

ENGENHARIA DE SOFTWARE · TRABALHO PRÁTICO

Cache Aside e Performance de APIs

Reduzindo latência e carga no banco com Redis

Demonstração prática do padrão **Cache Aside** com benchmark comparativo

Medição com 200 usuários virtuais · dois cenários isolados

Rails 7.2

PostgreSQL 16

Redis 7

Docker Compose

k6

Grafana · Loki

I. PROBLEMA

Consultas repetitivas sobrecarregam o banco

API acadêmica de catálogo (clientes, produtos, pedidos) com perfil **read-heavy**: muitas leituras, poucas escritas.

Sintomas sob carga

- Toda leitura vai direto ao PostgreSQL — sem atalho
- Dados de catálogo mudam pouco, mas são lidos com altíssima frequência
- O banco vira gargalo: a latência dispara e o throughput despenca
- Recursos do banco desperdiçados em consultas idênticas

✦ SOLUÇÃO PROPOSTA

Padrão Cache Aside

Uma camada de cache com **Redis** entre a API e o banco. Nos acertos (HIT), a resposta sai da memória — o banco nem é tocado.

↓ **Latência**

↑ **Throughput**

↓ **Carga DB**

I. PROBLEMA

O padrão Cache Aside: leitura e escrita

LEITURA

- 1. Consulta o **Redis**

HIT ✓

retorna direto
banco não é tocado

MISS

consulta o banco
grava no Redis e retorna

ESCRITA (create / update / delete)

- 1. Persiste no **banco**
- 2. **Invalida** a chave no Redis (apaga)
 - a próxima leitura vai ao banco e **renova** o cache

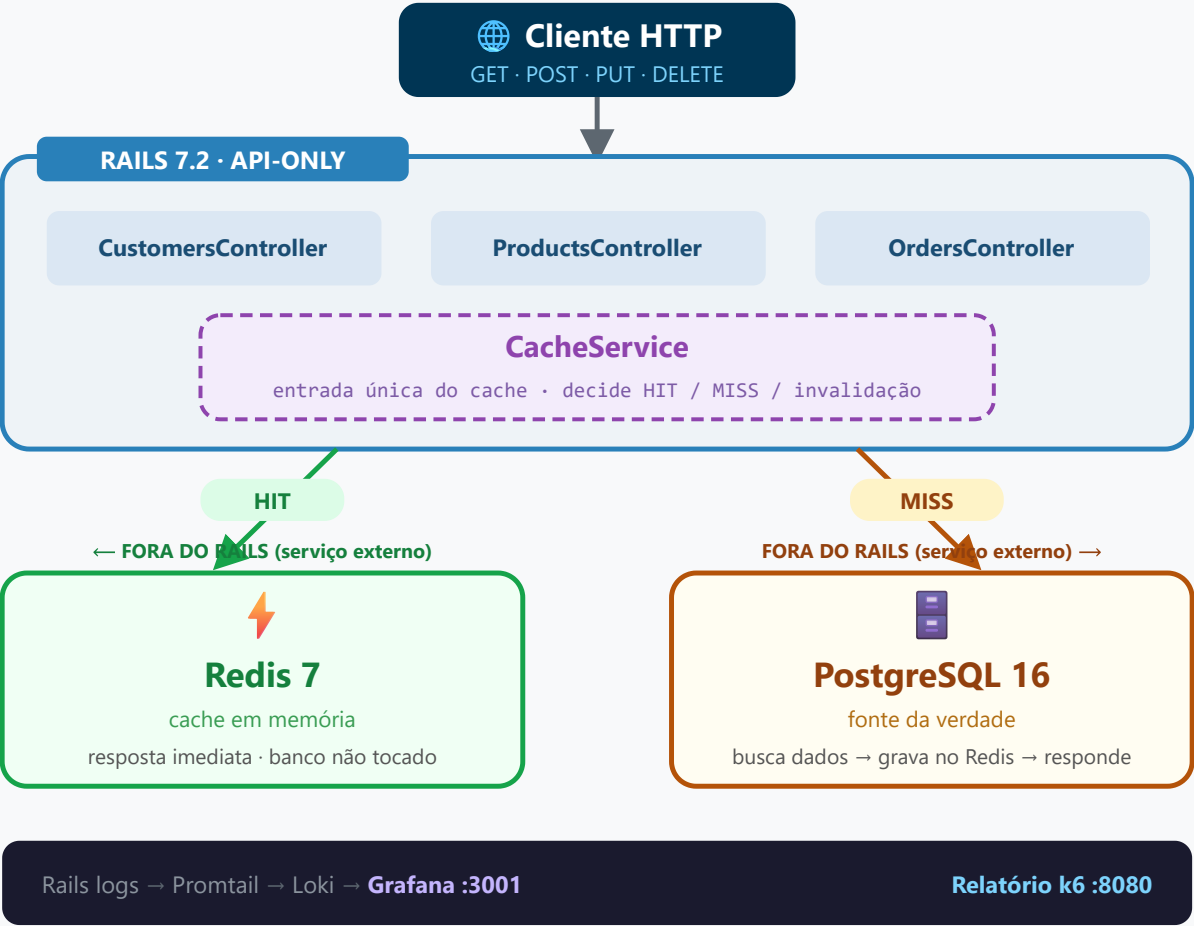
Por que cache-aside e não write-through?

A aplicação controla o cache explicitamente — sem biblioteca opaca, sem sincronização implícita. **Invalidar é mais simples e seguro do que atualizar**: elimina a janela de inconsistência, sem precisar garantir atomicidade na re-serialização do objeto.

II. ARQUITETURA

Arquitetura da solução

CACHE_ENABLED=true|false



Observabilidade: HIT / MISS / INVALIDATE rastreados nos logs

Fluxo em uma frase

O controller chama o **CacheService**; no **HIT** a resposta vem do Redis; no **MISS** consulta o PostgreSQL, grava no Redis e responde.

Chaves de cache

- customers:all · customers:<id>
- products:all · products:<id>
- orders:all · orders:<id>

Mesma imagem, dois cenários

A flag **CACHE_ENABLED** liga/desliga o cache sem rebuild — baseline vs tratamento.

III. RESULTADOS

Metodologia dos testes

Ferramenta: k6 (Grafana) — teste de carga com 200 usuários virtuais simultâneos · 40 req/s · ~220 s de duração.

JORNADA SIMULADA POR USUÁRIO

ETAPA	REQUISIÇÃO	DETALHE
Catálogo	GET /products	Lista todos os produtos
Produto	GET /products/:id	Detalha 1–2 produtos
Pedidos	GET /orders	JOIN pesado (pedidos + itens)
Compra	POST /orders (25%)	Cria pedido → invalida cache

Dataset (mesma seed nos dois cenários)

300 clientes · 500 produtos · 2.000 pedidos. Cache zerado (`FLUSHALL`) antes de cada run.

DOIS CENÁRIOS ISOLADOS

● Baseline — cache-off

`CACHE_ENABLED=false`

Toda requisição vai direto ao banco.

● Tratamento — cache-on

`CACHE_ENABLED=true`

Padrão cache-aside ativo · Redis zerado antes do run.

Condições idênticas entre cenários — a **única variável** é a presença do cache. Isso isola o efeito medido.

III. RESULTADOS

Resultados: latência

MÉTRICA	SEM CACHE	COM CACHE	MELHORA
Latência média	12.379 ms	5.471 ms	-55,8%
Latência p90	16.686 ms	7.461 ms	-55,3%
Latência p95	17.290 ms	7.713 ms	-55,4%
Latência p99	17.889 ms	8.170 ms	-54,3%
Latência máx.	18.786 ms	9.010 ms	-52,0%

O cache reduziu **~55% o tempo de resposta** em todos os percentis. O p95 caiu de 17,3 s para 7,7 s — ou seja, 95% das requisições ficaram abaixo de 7,7 s com cache.

~55%

redução de latência em todos os percentis

5,47 ms

média com cache

12,38 ms

média sem cache

III. RESULTADOS

Resultados: throughput e volume

MÉTRICA	SEM CACHE	COM CACHE	MELHORA
Requisições totais	2.982	6.044	+102,7%
Throughput (req/s)	13,54	28,14	+107,8%
Iterações concluídas	728	1.617	+122,1%
Iterações dropadas	5.450	4.657	-14,5%

Com cache, o servidor entregou **mais que o dobro de requisições** no mesmo tempo e completou **2,2× mais jornadas** de cliente.

+108%

throughput (req/s) com cache ativo

28,14

req/s com cache

13,54

req/s sem cache

III. RESULTADOS

Cache hit, miss e invalidação

Medido durante o cenário `cache-on` com 200 usuários virtuais:

CONTADOR	VALOR
Cache HITs	420
Cache MISSES	291
Hit Rate	59,1%
Invalidações	49

As **49 invalidações** correspondem às compras (`POST /orders`) — cada compra apaga `orders:all` e renova o cache na próxima leitura.

CONSULTAS AO BANCO

CENÁRIO	CONSULTAS	DIFERENÇA
Sem cache	1.422	base
Com cache	804	-43,5%

43,5%

menos consultas ao PostgreSQL

IV. ANÁLISE

Consistência dos dados

GARANTIA	ESTE PROJETO
Consistência forte	✗ Não garante
Consistência eventual	✓ Converge rápido
Defasagem máxima	≤ 300 s (TTL)
Defasagem típica	próxima requisição

Estratégia: invalidate-on-write

Escrita → banco persistido → chave **apagada** (não atualizada). Não há janela de dado desatualizado *ativo* no Redis — a chave deixa de existir e a próxima leitura renova.

Condição de corrida (benigna)

Duas leituras simultâneas no MISS fazem duas consultas — ambas gravam o **mesmo** dado no Redis. Inofensivo. A gem `redis-client` é thread-safe.

TTL (300 s) como rede de segurança

Se uma invalidação falhar (Redis momentaneamente indisponível), a chave expira em 5 min e o sistema volta ao estado correto — sem intervenção manual.

Aceitamos leitura eventualmente desatualizada na janela de invalidação em troca de ~2× throughput e ~55% menos latência — aceitável para dados de catálogo.

IV. ANÁLISE

Trade-offs arquiteturais

DIMENSÃO	SEM CACHE	COM CACHE
Latência	Alta (banco toda vez)	Baixa nos HITs (~55% ↓)
Throughput	Limitado pelo banco	~2× maior
Consistência	Forte	Eventual (janela mín.)
Complexidade	Simples	Maior (invalidação + TTL)
Resiliência	Ponto único: banco	Redis off → fallback ao banco
Memória	Zero extra	Redis ~MB de catálogo
Warm-up	Não necessário	1ªs requisições são MISS

Quando cache-aside faz sentido

- Leituras >> escritas (**read-heavy**)
- Dados com baixa volatilidade (catálogo)
- Tolerância a consistência eventual
- Latência de banco acima do aceitável sob carga

Fora desse perfil (escrita pesada, consistência forte obrigatória), o cache adiciona complexidade sem ganho proporcional.

IV. ANÁLISE

Conclusão

OBJETIVOS DO ENUNCIADO × RESULTADO

OBJETIVO	RESULTADO MEDIDO
Reduzir latência	–55% em todos os percentis
Diminuir carga no banco	–43,5% de consultas
Melhorar desempenho	+108% de throughput
Demonstrar hit/miss	59,1% · 49 invalidações
Discutir consistência	Eventual · documentada

LIÇÕES ARQUITETURAIS

1. Cache-aside dá **controle total** — e responsabilidade total — à aplicação.
2. **Invalidar** é mais seguro que atualizar: elimina dado inconsistente ativo no Redis.
3. O TTL não é só otimização de memória — é **rede de segurança** para falhas de invalidação.
4. 59% de hit rate já entrega ganho expressivo; sistemas reais com dados estáveis atingem **80–95%**.



Com o padrão Cache Aside, a mesma API entregou **o dobro de throughput** e **metade da latência**, custando ao banco **menos da metade** das consultas — ao preço de consistência eventual controlada.